

Practice Project: Create an Expense Tracking App

Estimated time: 60 minutes



Introduction

Welcome to this practice project, where you will develop an Expense Management application using Flutter. This project helps you apply the skills and knowledge acquired throughout the course.

The application you create will allow the user to manage their expenses with a friendly and intuitive user experience (UX). You will implement various essential features for an expense management app, such as adding, viewing, filtering, and deleting expenses.

Additionally, you will ensure that the data persists locally so that the user's expenses are not lost when the app is closed and reopened. We will use local storage for this feature.

User stories

The following user stories outline the key functionalities that you need to implement:

- "I want to view a list of all my expenses to see my spending history at a glance."
- "I want to add an expense with a payee, amount, notes, date, category, and tag to keep track of my spending."
- "I want to save my expenses locally so I do not lose them when I close and reopen the app."
- "I want to group my expenses by category to see how much I spend in different areas."
- "I want to delete an expense to remove any incorrect or unnecessary entries."
- "I want to manage categories and tags in the app settings."

Objectives

After completing this practice project, you will be able to:

- Create an intuitive UX for the user to list expenses
- Give the user a way to add and delete expenses
- Give the user a way to manage categories and tags
- Implement CRUD operations for expense data
- Integrate local storage to save data persistently

About Cloud IDE

Running the lab

This lab is designed to be completed using the Skills Network Cloud IDE environment. You will write and execute Flutter code directly within the IDE.

About Skills Network Cloud IDE

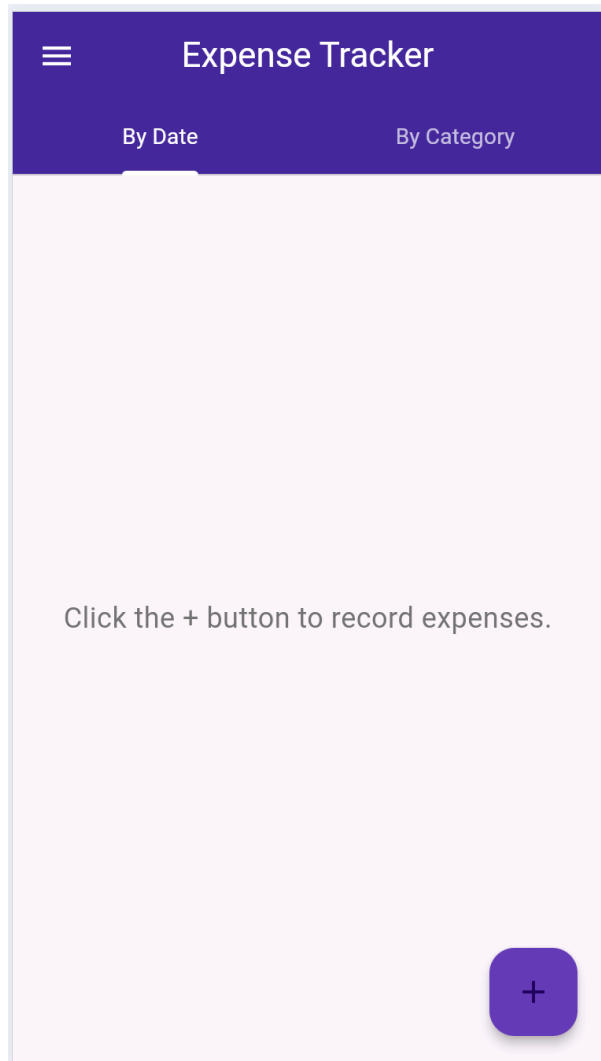
Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands-on labs for course and project-related labs. It is an open-source Integrated Development Environment (IDE).

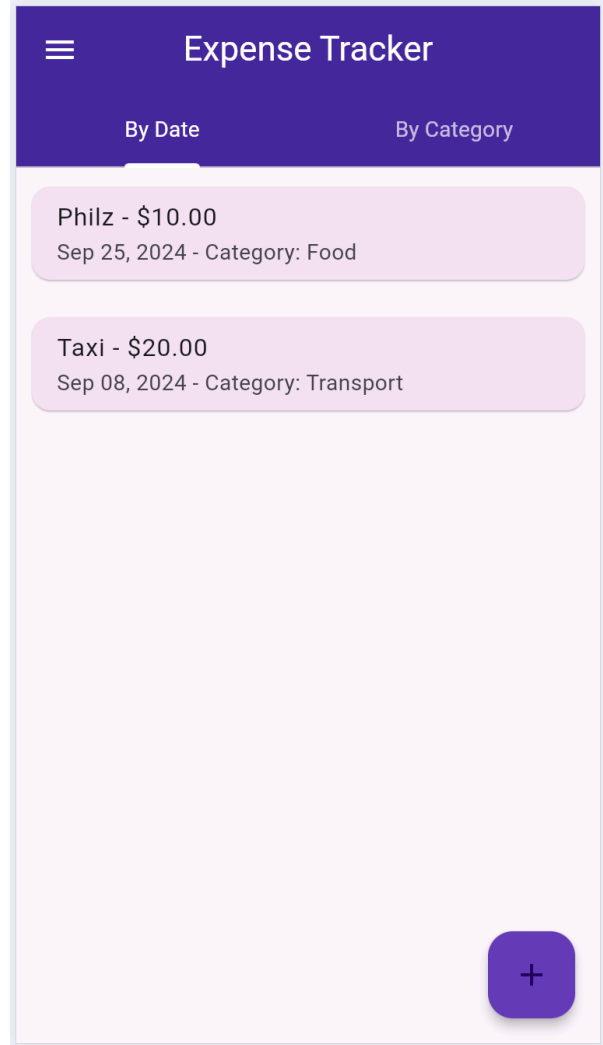
Important notice about this lab environment

Please be aware that sessions for this lab environment are not persistent. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session will be lost. Plan to complete these labs in a single session to avoid losing your data.

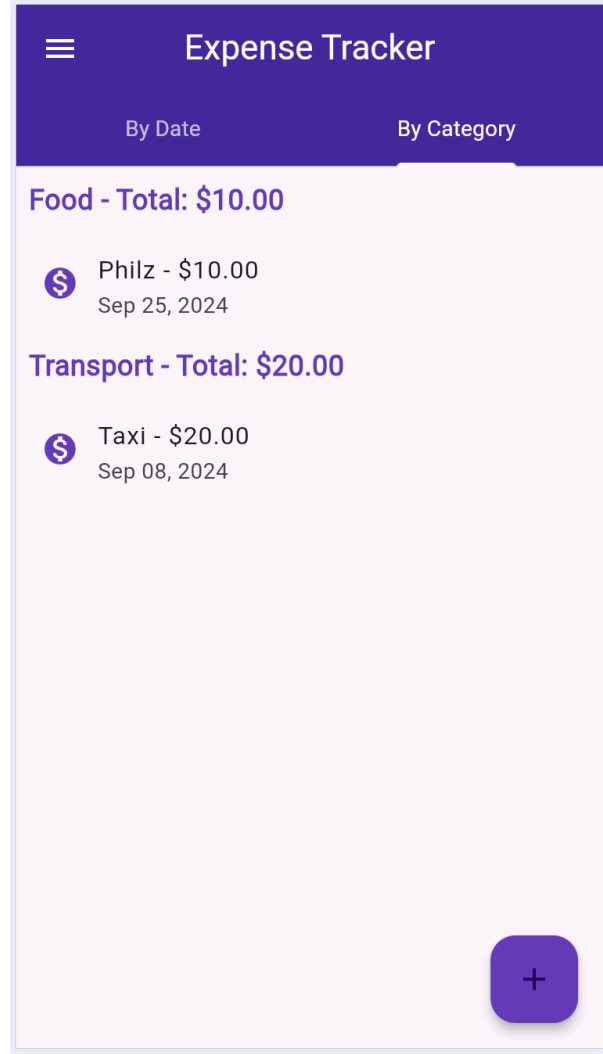
Final application screens

- Empty home screen when the user launches the application





- Home screen showing all expenses by category



- Screen to add a new expense

←

Add Expense

Amount

10

Payee

Philz

note

Morning coffee

Date: 2024-09-25



Category

Food

▼

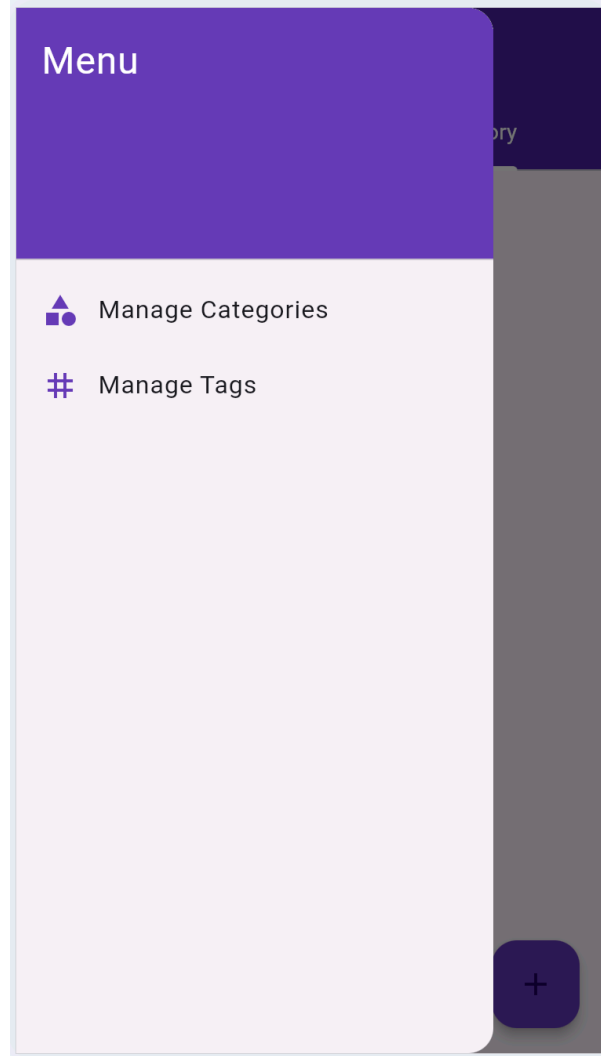
Tag

Cafe

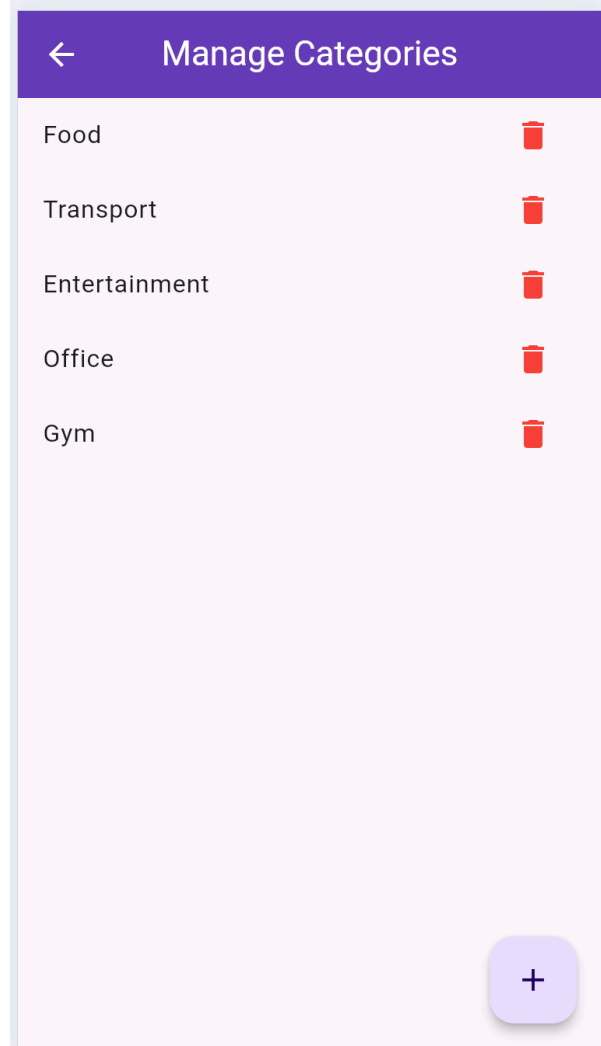
▼

Save Expense

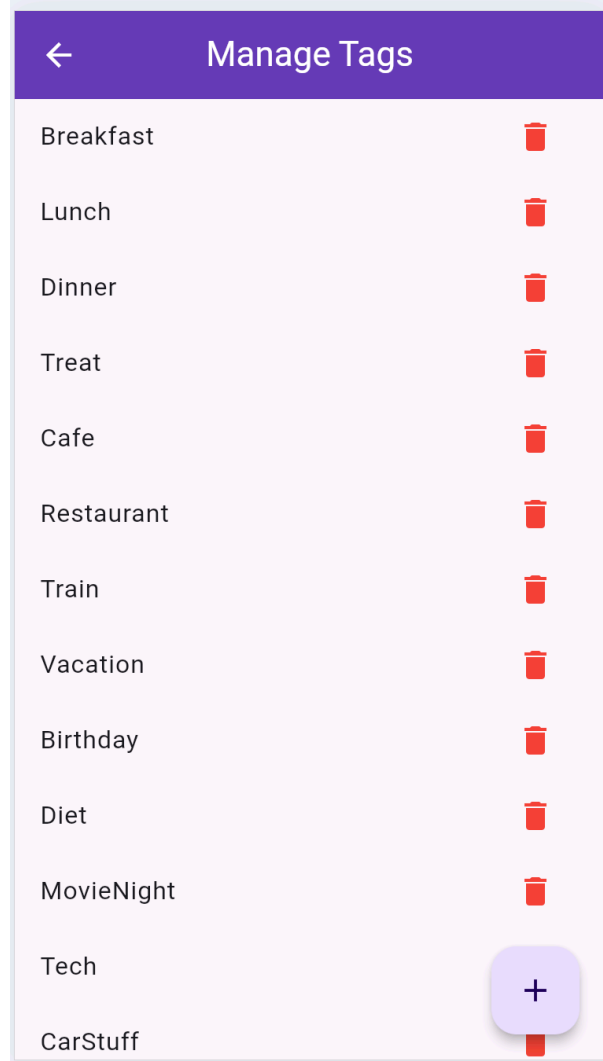
- Hamburger menu to manage tags and categories



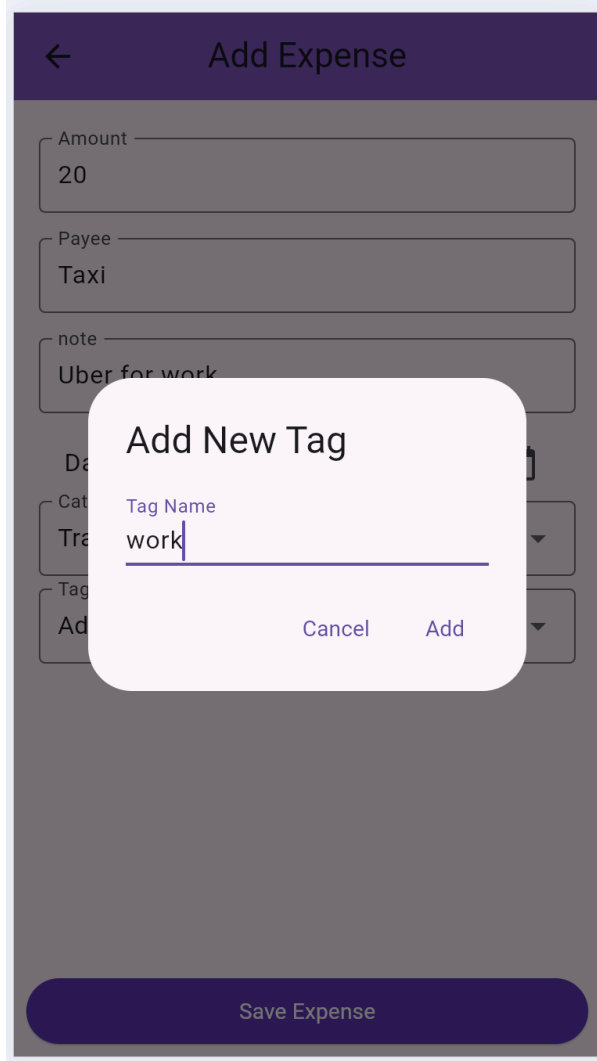
- Screen to manage categories



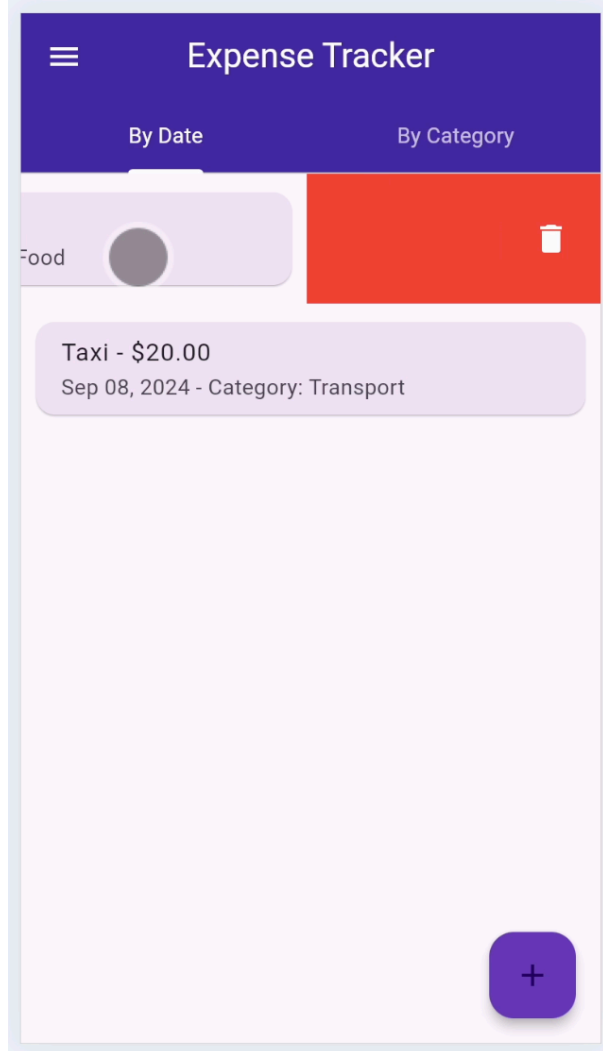
- Screen to manage tags



- Adding a new tag

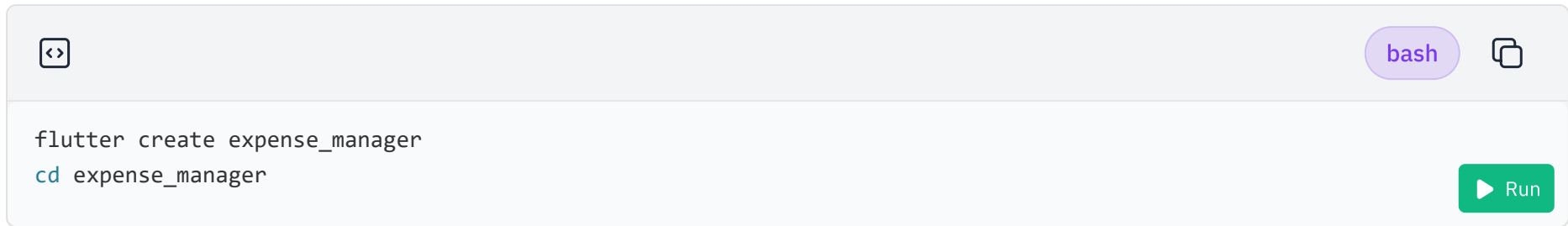


- Deleting an expense by swiping left



Step 1: Create the Flutter project

- You will start by creating a new Flutter project using the command line provided by the Cloud IDE.
- This step sets up the project with all necessary Flutter files and directory structure for developing the application.



The screenshot shows a terminal window with a light blue header bar containing a code icon, a 'bash' button, and a copy icon. The terminal area has a white background and contains the following commands: `flutter create expense_manager` and `cd expense_manager`. A green 'Run' button with a play icon is located at the bottom right of the terminal area.

- `flutter create expense_manager`: This command creates a new Flutter project named `expense_manager`.
- `cd expense_manager`: Change the directory to the newly created project folder to start developing the application.

Note that the rest of the commands in the lab might not work if you do not use the same application name `expense_manager`. If you decide to use a different name, make sure to also change the commands that refer to the application name or directory.

Install dependencies

You will need the following packages:

- **intl**: Used for date and number formatting and parsing.
- **provider**: A wrapper around `InheritedWidget` to make them easier to use and more reusable.
- **collection**: Classes and utilities that supplement the collection support in Dart.

Use the following commands to install these packages:

bash

```
flutter pub add intl  
flutter pub add provider  
flutter pub add collection  
flutter pub add localstorage
```

Run

Step 2: Define the models

In this step, you will define all the models required for the expense management application. These models will help structure the data used throughout the application.

You'll create an `Expense` class, a `ExpenseCategory` class, and a `Tag` class with their respective properties.

Part 1: Create the Expense class with properties

First, create the `Expense` class and define its properties.



dart



```
class Expense {  
  final String id;  
  final double amount;  
  final String categoryId;  
  final String payee;  
  final String note;  
  final DateTime date;  
  final String tag;  
  
  Expense({  
    required this.id,  
    required this.amount,  
    required this.categoryId,  
    required this.payee,  
    required this.note,  
    required this.date,  
    required this.tag,  
  });  
}
```

- `class Expense` : Defines a class named `Expense`.
- `final` : Fields are immutable.
- `Expense({ ... })` : Constructor to initialize the properties.

Part 2: Add the fromJson method to Expense

Next, add a factory constructor to create an `Expense` instance from a JSON object.



dart



```
class Expense {  
  // ... (previous code)  
  
  factory Expense.fromJson(Map<String, dynamic> json) {  
    return Expense(  
      id: json['id'],  
      amount: json['amount'],  
      categoryId: json['categoryId'],  
      payee: json['payee'],  
      note: json['note'],  
      date: DateTime.parse(json['date']),  
      tag: json['tag'],  
    );  
  }  
}
```

- `factory Expense.fromJson`: A factory constructor for creating an `Expense` instance from a map.

Part 3: Add the toJson method to Expense

Finally, add a method to convert an `Expense` instance to a JSON object.



dart



```
class Expense {  
  // ... (previous code)  
  
  Map<String, dynamic> toJson() {  
    return {  
      'id': id,  
      'amount': amount,  
      'categoryId': categoryId,  
      'payee': payee,  
      'note': note,  
      'date': date.toIso8601String(),  
      'tag': tag,  
    };  
  }  
}
```

- `Map<String, dynamic> toJson()`: Method to convert an `Expense` instance to a map.

Part 4: Create the ExpenseCategory class

Next, create the ExpenseCategory class



dart



```
class ExpenseCategory {  
  final String id;  
  final String name;  
  final bool isDefault;  
  
  ExpenseCategory({  
    required this.id,  
    required this.name,  
    this.isDefault = false,  
  });  
}
```

- `class ExpenseCategory`: Defines a class named `ExpenseCategory`.
- `final`: Fields are immutable.
- `ExpenseCategory{ ... }`: Constructor to initialize the properties.
- `isDefault`: Indicates whether the category is system default or user-added.

Part 5: Add the fromJson method to ExpenseCategory

Add a factory constructor to create a `ExpenseCategory` instance from a JSON object.



dart



```
class ExpenseCategory {  
  // ... (previous code)  
  
  factory ExpenseCategory.fromJson(Map<String, dynamic> json) {  
    return ExpenseCategory(  
      id: json['id'],  
      name: json['name'],  
      isDefault: json['isDefault'] ?? false,  
    );  
  }  
}
```

- **factory ExpenseCategory.fromJson**: A factory constructor for creating a **ExpenseCategory** instance from a map.

Part 6: Add the toJson method to ExpenseCategory

Add a method to convert a **ExpenseCategory** instance to a JSON object.



dart



```
class ExpenseCategory {  
  // ... (previous code)  
  
  Map<String, dynamic> toJson() {  
    return {  
      'id': id,  
      'name': name,  
      'isDefault': isDefault,  
    };  
  }  
}
```

- `Map<String, dynamic> toJson()` : Method to convert a `ExpenseCategory` instance to a map.
- `isDefault` is used to indicate whether an `ExpenseCategory` is a system-defined category or a user-created category, and the `= false` part sets a default value.

Part 7: Create the Tag class

Finally, create the `Tag` class and define its properties.



dart



```
class Tag {  
  final String id;  
  final String name;  
  
  Tag({  
    required this.id,  
    required this.name,  
  });  
}
```

- `class Tag`: Defines a class named `Tag`.
- `final`: Fields are immutable.
- `Tag({ ... })`: Constructor to initialize the properties.

Part 8: Add the fromJson method to Tag

Add a factory constructor to create a `Tag` instance from a JSON object.



dart



```
class Tag {  
  // ... (previous code)  
  
  factory Tag.fromJson(Map<String, dynamic> json) {  
    return Tag(  
      id: json['id'],  
      name: json['name'],  
    );  
  }  
}
```

- `factory Tag.fromJson`: A factory constructor for creating a `Tag` instance from a map.

Part 9: Add the toJson method to Tag

Add a method to convert a `Tag` instance to a JSON object.



dart



```
class Tag {  
  // ... (previous code)  
  
  Map<String, dynamic> toJson() {  
    return {  
      'id': id,  
      'name': name,  
    };  
  }  
}
```

- `Map<String, dynamic> toJson()` : Method to convert a `Tag` instance to a map.

Step 3: Create the expense provider

This step involves creating a provider for managing the state of expense data in your app. The provider handles expenses, categories, and tags, including adding, updating, deleting, and storing expenses locally.

Part 1: Create the ExpenseProvider class

First, create the `ExpenseProvider` class and define its properties. This provider is responsible for managing all expense-related state in the application. It stores expenses, categories, and tags in memory and persists expenses using `LocalStorage`. It also notifies the UI whenever data changes using `ChangeNotifier`.



dart



```
import 'package:flutter/material.dart';
import '../models/expense.dart';
import '../models/expense_category.dart';
import '../models/tag.dart';
import 'package:localstorage/localstorage.dart';
import 'dart:convert';

class ExpenseProvider with ChangeNotifier {
  final LocalStorage storage;
  // List of expenses
  List<Expense> _expenses = [];
  // List of categories
  final List<ExpenseCategory> _categories = [
    ExpenseCategory(id: '1', name: 'Food', isDefault: true),
    ExpenseCategory(id: '2', name: 'Transport', isDefault: true),
    ExpenseCategory(id: '3', name: 'Entertainment', isDefault: true),
    ExpenseCategory(id: '4', name: 'Office', isDefault: true),
    ExpenseCategory(id: '5', name: 'Gym', isDefault: true),
  ];
  // List of tags
  final List<Tag> _tags = [
    Tag(id: '1', name: 'Breakfast'),
    Tag(id: '2', name: 'Lunch'),
    Tag(id: '3', name: 'Dinner'),
    Tag(id: '4', name: 'Treat'),
    Tag(id: '5', name: 'Cafe'),
    Tag(id: '6', name: 'Restaurant'),
    Tag(id: '7', name: 'Train'),
    Tag(id: '8', name: 'Vacation'),
    Tag(id: '9', name: 'Birthday'),
    Tag(id: '10', name: 'Diet'),
    Tag(id: '11', name: 'MovieNight'),
  ];
}
```

```

    Tag(id: '14', name: 'SelfCare'),
    Tag(id: '15', name: 'Streaming'),
  ];
  // Getters
  List<Expense> get expenses => _expenses;
  List<ExpenseCategory> get categories => _categories;
  List<Tag> get tags => _tags;
  ExpenseProvider(this.storage) {
    _loadExpensesFromStorage();
  }
}

```

- `class ExpenseProvider with ChangeNotifier` : A class to manage the list of expenses.
- `List<ExpenseCategory> _categories` : A private list to store user categories
- `List<Tag> _tags` : A private list to store labeling expenses
- `List<Expense> _expenses` : A private list to store expenses.
- `List<Expense> get expenses` : A getter to access the list of expenses.

Part 2: Load expenses from local storage

Next, add a method to load expenses from local storage.



dart



```
void _loadExpensesFromStorage() {  
  var storedExpenses = storage.getItem('expenses');  
  if (storedExpenses != null) {  
    final List<dynamic> decoded = jsonDecode(storedExpenses as String);  
    _expenses = decoded  
      .map((item) => Expense.fromJson(Map<String, dynamic>.from(item)))  
      .toList();  
    notifyListeners();  
  }  
}
```

- `_loadExpensesFromStorage()`: This method retrieves saved expenses from local storage when the provider is initialized.

Part 3: Save expenses to local storage

Next, add a method to save expenses to local storage.



dart



```
void _saveExpensesToStorage() {  
  storage.setItem(  
    'expenses',  
    jsonEncode(_expenses.map((e) => e.toJson()).toList()),  
  );  
}
```

Part 4: Add expense methods

Add methods to add, update, and remove expenses.

 dart 

```
void addExpense(Expense expense) {
  _expenses.add(expense);
  _saveExpensesToStorage();
  notifyListeners();
}

void addOrUpdateExpense(Expense expense) {
  int index = _expenses.indexWhere((e) => e.id == expense.id);
  if (index != -1) {
    // Update existing expense
    _expenses[index] = expense;
  } else {
    // Add new expense
    _expenses.add(expense);
  }
  _saveExpensesToStorage(); // Save the updated list to local storage
  notifyListeners();
}

void removeExpense(String id) {
  _expenses.removeWhere((expense) => expense.id == id);
  _saveExpensesToStorage();
  notifyListeners();
}
```

- `addExpense(Expense expense)` : Method to add a new expense.

- `removeExpense(String id)`: Method to remove an expense.

Part 5: Category management

Add methods to manage expense categories.



dart



```
void addCategory(ExpenseCategory category) {  
  if (!_categories.any((cat) => cat.name == category.name)) {  
    _categories.add(category);  
    notifyListeners();  
  }  
}  
  
void deleteCategory(String id) {  
  _categories.removeWhere((category) => category.id == id);  
  notifyListeners();  
}
```

Part 6: Tag management

Add methods to manage tags.



dart



```
void addTag(Tag tag) {  
  if (!_tags.any((t) => t.name == tag.name)) {  
    _tags.add(tag);  
    notifyListeners();  
  }  
}  
  
void deleteTag(String id) {  
  _tags.removeWhere((tag) => tag.id == id);  
  notifyListeners();  
}
```

Step 4: Add the widgets

In this step, you will create the necessary widgets for the expense management application. You'll create an `AddCategoryDialog` widget and an `AddTagDialog` widget to handle adding new categories and tags.

Part 1: Create the AddCategoryDialog widget

First, create the `AddCategoryDialog` widget.



dart



```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/expense_category.dart';
import '../providers/expense_provider.dart';

class AddCategoryDialog extends StatefulWidget {
  final Function(ExpenseCategory) onAdd;

  AddCategoryDialog({required this.onAdd});

  @override
  _AddCategoryDialogState createState() => _AddCategoryDialogState();
}

class _AddCategoryDialogState extends State<AddCategoryDialog> {
  final TextEditingController _controller = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text('Add New Category'),
      content: TextField(
        controller: _controller,
        decoration: InputDecoration(
          labelText: 'Category Name',
        ),
      ),
      actions: [
        TextButton(
          child: Text('Cancel'),
          onPressed: () => Navigator.of(context).pop(),
        ),
      ],
    );
  }
}
```

```

        onPressed: () {
          var newCategory = ExpenseCategory(
            id: DateTime.now().toString(), name: _controller.text);
          widget.onAdd(newCategory);
          Provider.of<ExpenseProvider>(context, listen: false)
            .addCategory(newCategory);
          _controller.clear(); // Clear the input field
          Navigator.of(context).pop();
        },
      ),
    ],
  );
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
}

```

- `class AddCategoryDialog extends StatefulWidget`: Defines a stateful widget named `AddCategoryDialog`.
- Uses `TextEditingController` to manage the input for the category name.
- Builds an `AlertDialog` with a text field and action buttons.

Part 2: Create the AddTagDialog widget

Next, create the `AddTagDialog` widget.



dart



```
import 'package:provider/provider.dart';
import 'package:flutter/material.dart';
import '../models/tag.dart';
import '../providers/expense_provider.dart';

class AddTagDialog extends StatefulWidget {
  final Function(Tag) onAdd;

  AddTagDialog({required this.onAdd});

  @override
  _AddTagDialogState createState() => _AddTagDialogState();
}

class _AddTagDialogState extends State<AddTagDialog> {
  final TextEditingController _controller = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text('Add New Tag'),
      content: TextField(
        controller: _controller,
        decoration: InputDecoration(
          labelText: 'Tag Name',
        ),
      ),
      actions: [
        TextButton(
          child: Text('Cancel'),
          onPressed: () => Navigator.of(context).pop(),
        ),
      ],
    );
  }
}
```

```

        onPressed: () {
          var newTag = Tag(id: DateTime.now().toString(), name: _controller.text);
          widget.onAdd(newTag);
          // Update the provider and UI
          Provider.of<ExpenseProvider>(context, listen: false).addTag(newTag);
          // Clear the input field for next input
          _controller.clear();

          Navigator.of(context).pop();
        },
      ),
    ],
  );
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
}

```

- `class AddTagDialog extends StatefulWidget`: Defines a stateful widget named `AddTagDialog`.
- Uses `TextEditingController` to manage the input for the tag name.
- Builds an `AlertDialog` with a text field and action buttons.

Step 5: Create the add expense screen

This step involves creating a screen to add expense details. You will implement form fields to capture expense information. This step involves creating a screen where users can add or edit expense details. The screen includes form inputs, dropdowns for category and tag, and a date picker.

This version supports both adding new expenses and editing existing ones. If `initialExpense` is null, the screen works in "Add mode". If `initialExpense` is provided, the screen works in "Edit mode" and pre-fills all fields.

Part 1: Create the `AddExpenseScreen` class

First, create the `AddExpenseScreen` class and define its properties and state.



dart



```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';

import '../models/expense.dart';
import '../providers/expense_provider.dart';
import '../widgets/add_category_dialog.dart';
import '../widgets/add_tag_dialog.dart';

class AddExpenseScreen extends StatefulWidget {
  final Expense? initialExpense; // Used for editing existing expense

  const AddExpenseScreen({Key? key, this.initialExpense}) : super(key: key);

  @override
  _AddExpenseScreenState createState() => _AddExpenseScreenState();
}
```

- `class AddExpenseScreen extends StatefulWidget`: Defines a stateful widget named `AddExpenseScreen`.
- `initialExpense` allows edit mode support
 - If null → new expense
 - If not null → edit existing expense

Part 2: Initialize form controllers and state

Define controllers and selected values for the form.



dart



```
class _AddExpenseScreenState extends State<AddExpenseScreen> {  
  late TextEditingController _amountController;  
  late TextEditingController _payeeController;  
  late TextEditingController _noteController;  
  
  String? _selectedCategoryId;  
  String? _selectedTagId;  
  DateTime _selectedDate = DateTime.now();  
}
```

- Controllers handle text input
- Selected IDs store dropdown selection
- Default date is today

Part 3: Initialize values in initState

Pre-fill fields when editing an expense.



dart



```
@override
void initState() {
  super.initState();

  _amountController = TextEditingController(
    text: widget.initialExpense?.amount.toString() ?? '',
  );

  _payeeController = TextEditingController(
    text: widget.initialExpense?.payee ?? '',
  );

  _noteController = TextEditingController(
    text: widget.initialExpense?.note ?? '',
  );

  _selectedDate = widget.initialExpense?.date ?? DateTime.now();
  _selectedCategoryId = widget.initialExpense?.categoryId;
  _selectedTagId = widget.initialExpense?.tag;
}
```

- Automatically fills the form in edit mode
- Keeps an empty form for a new expense

Part 4: Build the UI

The build method constructs the UI and connects the screen to the ExpenseProvider so it can access categories, tags, and save data.



dart



```
@override
Widget build(BuildContext context) {
  final expenseProvider = Provider.of<ExpenseProvider>(context);

  return Scaffold(
    appBar: AppBar(
      title: Text(
        widget.initialExpense == null ? 'Add Expense' : 'Edit Expense',
      ),
      backgroundColor: Colors.deepPurple[400],
    ),
```

- Title changes based on add/edit mode
- Provider gives access to categories, tags, and expenses

Part 5:Form UI section



dart



```
body: SingleChildScrollView(
  padding: EdgeInsets.all(16),
  child: Column(
    children: [
      buildTextField(_amountController, 'Amount',
        TextInputType.numberWithOptions(decimal: true)),

      buildTextField(_payeeController, 'Payee', TextInputType.text),
      buildTextField(_noteController, 'Note', TextInputType.text),

      buildDateField(_selectedDate),

      Padding(
        padding: const EdgeInsets.only(bottom: 8.0),
        child: buildCategoryDropdown(expenseProvider),
      ),

      Padding(
        padding: const EdgeInsets.only(bottom: 8.0),
        child: buildTagDropdown(expenseProvider),
      ),
    ],
  ),
),
```

- Uses reusable helper widgets
- Includes: amount input, payee input, note input, date picker, category dropdown, tag dropdown

Part 6: Save button



dart



```
bottomNavigationBar: Padding(  
  padding: EdgeInsets.all(8.0),  
  child: ElevatedButton(  
    style: ElevatedButton.styleFrom(  
      backgroundColor: Colors.deepPurple,  
      foregroundColor: Colors.white,  
      minimumSize: Size(double.infinity, 50),  
    ),  
    onPressed: _saveExpense,  
    child: Text('Save Expense'),  
  ),  
,  
);  
}
```

- The Save button triggers validation and either creates a new expense or updates an existing one using the provider.

Part 7: Save expense logic

Create or update expense and send it to provider.



dart



```
void _saveExpense() {
  if (_amountController.text.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Please fill in required fields!')),
    );
    return;
  }

  final expense = Expense(
    id: widget.initialExpense?.id ?? DateTime.now().toString(),
    amount: double.parse(_amountController.text),
    categoryId: _selectedCategoryId!,
    payee: _payeeController.text,
    note: _noteController.text,
    date: _selectedDate,
    tag: _selectedTagId!,
  );

  Provider.of<ExpenseProvider>(context, listen: false)
    .addOrUpdateExpense(expense);

  Navigator.pop(context);
}
```

- Validates required fields
- Creates Expense object
- Uses addOrUpdateExpense (handles both add + edit)
- Returns to previous screen

Part 8: Helper method — text field

Reusable input field builder.



```
Widget buildTextField(  
  TextEditingController controller,  
  String label,  
  TextInputType type,  
) {  
  return Padding(  
    padding: const EdgeInsets.symmetric(vertical: 8.0),  
    child: TextField(  
      controller: controller,  
      keyboardType: type,  
      decoration: InputDecoration(  
        labelText: label,  
        border: OutlineInputBorder(),  
      ),  
    ),  
  );  
}
```

- Reduces repetition
- Standard UI styling for inputs

Part 9: Date picker field

Allows user to select a date.



dart



```
Widget buildDateField(DateTime selectedDate) {  
  return ListTile(  
    title: Text(  
      "Date: ${DateFormat('yyyy-MM-dd').format(selectedDate)}",  
    ),  
    trailing: Icon(Icons.calendar_today),  
    onTap: () async {  
      final picked = await showDatePicker(  
        context: context,  
        initialDate: selectedDate,  
        firstDate: DateTime(2000),  
        lastDate: DateTime(2100),  
      );  
  
      if (picked != null) {  
        setState(() {  
          _selectedDate = picked;  
        });  
      }  
    },  
  );  
}
```

Part 10: Category dropdown

Dropdown with option to add new category.



dart



```
Widget buildCategoryDropdown(ExpenseProvider provider) {  
  return DropdownButtonFormField<String>(  
    value: _selectedCategoryId,  
    onChanged: (newValue) {  
      if (newValue == 'New') {  
        showDialog(  
          context: context,  
          builder: (context) => AddCategoryDialog(  
            onAdd: (newCategory) {  
              setState(() {  
                _selectedCategoryId = newCategory.id;  
              });  
  
              provider.addCategory(newCategory);  
            },  
          ),  
        );  
      } else {  
        setState(() => _selectedCategoryId = newValue);  
      }  
    },  
  ),  
);
```

- If the user selects "Add New Category", a dialog opens allowing them to create a new item dynamically.



dart



```
items: provider.categories
  .map<DropdownMenuItem<String>>((category) {
    return DropdownMenuItem(
      value: category.id,
      child: Text(category.name),
    );
  }).toList()
  ..add(
    DropdownMenuItem(
      value: "New",
      child: Text("Add New Category"),
    ),
  ),
decoration: InputDecoration(
  labelText: 'Category',
  border: OutlineInputBorder(),
),
);
}
```

Part 11: Tag dropdown

Same logic as category dropdown.



dart



```
Widget buildTagDropdown(ExpenseProvider provider) {  
  return DropdownButtonFormField<String>(  
    value: _selectedTagId,  
    onChanged: (newValue) {  
      if (newValue == 'New') {  
        showDialog(  
          context: context,  
          builder: (context) => AddTagDialog(  
            onAdd: (newTag) {  
              provider.addTag(newTag);  
  
              setState(() {  
                _selectedTagId = newTag.id;  
              });  
            },  
          ),  
        );  
      } else {  
        setState(() => _selectedTagId = newValue);  
      }  
    },  
  ),  
);
```

- If the user selects "Add New Tag", a dialog opens allowing them to create a new item dynamically.



dart



```
items: provider.tags
  .map<DropdownMenuItem<String>>((tag) {
    return DropdownMenuItem(
      value: tag.id,
      child: Text(tag.name),
    );
  }).toList()
  ..add(
    DropdownMenuItem(
      value: "New",
      child: Text("Add New Tag"),
    ),
  ),
decoration: InputDecoration(
  labelText: 'Tag',
  border: OutlineInputBorder(),
),
);
}
```

The `add expense` screen should look like this at this time:

Add Expense

Amount

10

Payee

Philz

note

Morning coffee

Date: 2024-09-25



Category

Food

Tag

Cafe

Save Expense

Step 6: Create the category management screen

This step involves creating a screen to manage expense categories. You will implement functionality to add and delete categories.

Part 1: Create the CategoryManagementScreen class

First, create the `CategoryManagementScreen` class.



dart



```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/expense_category.dart';
import '../providers/expense_provider.dart';
import '../widgets/add_category_dialog.dart';

// Example for CategoryManagementScreen
class CategoryManagementScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Manage Categories"),
        backgroundColor: Colors.deepPurple, // Themed color similar to your inspirations
        foregroundColor: Colors.white,
      ),
      body: Consumer<ExpenseProvider>(
        builder: (context, provider, child) {
          return ListView.builder(
            itemCount: provider.categories.length,
            itemBuilder: (context, index) {
              final category = provider.categories[index];
              return ListTile(
                title: Text(category.name),
                trailing: IconButton(
                  icon: Icon(Icons.delete, color: Colors.red),
                  onPressed: () {
                    provider.deleteCategory(category.id);
                  },
                ),
              );
            },
          );
        },
      ),
    );
  }
}
```

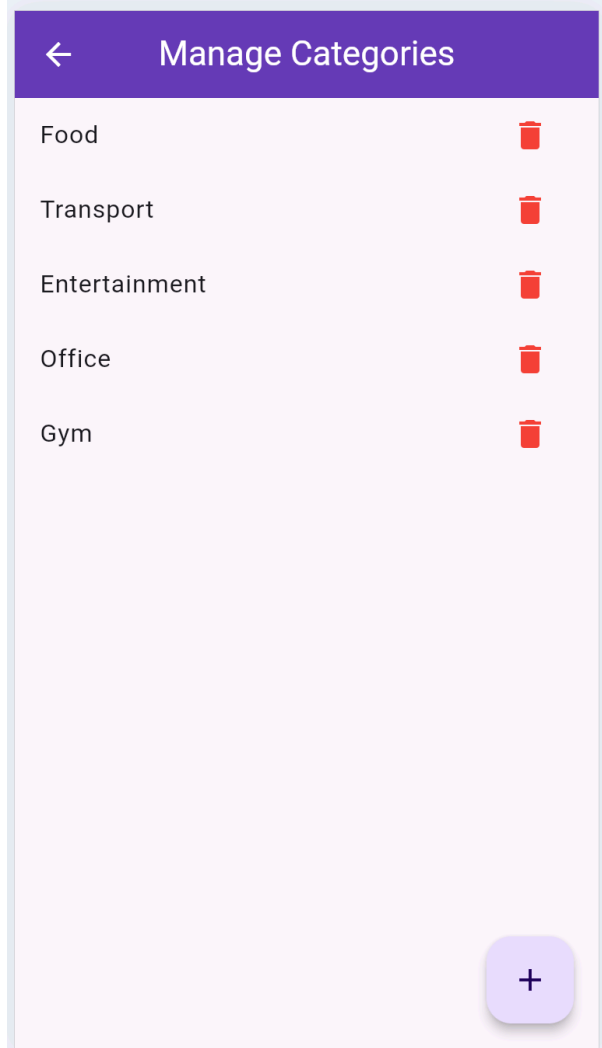
```

    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () {
        showDialog(
          context: context,
          builder: (context) => AddCategoryDialog(
            onAdd: (newCategory) {
              Provider.of<ExpenseProvider>(context, listen: false)
                .addCategory(newCategory);
              Navigator.pop(context); // Close the dialog
            },
          ),
        );
      },
      tooltip: 'Add New Category',
      child: Icon(Icons.add),
    ),
  );
}
}

```

- `class CategoryManagementScreen extends StatelessWidget`: Defines a stateless widget named `CategoryManagementScreen`.
- Uses `Consumer<ExpenseProvider>` to listen to category list updates.
- Implements a `ListView.builder` to render each category as a list item.
- Defines a floating action button to show a dialog for adding a new category.

The `category management` screen should look like this:



Step 7: Create the tag management screen

This step involves creating a screen to manage expense tags. You will implement functionality to add and delete tags.

Create the TagManagementScreen class

First, create the `TagManagementScreen` class.



dart



```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/expense_provider.dart';
import '../widgets/add_tag_dialog.dart';

class TagManagementScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Manage Tags"),
        backgroundColor: Colors.deepPurple, // Themed color similar to your inspirations
        foregroundColor: Colors.white,
      ),
      body: Consumer<ExpenseProvider>(
        builder: (context, provider, child) {
          return ListView.builder(
            itemCount: provider.tags.length,
            itemBuilder: (context, index) {
              final tag = provider.tags[index];
              return ListTile(
                title: Text(tag.name),
                trailing: IconButton(
                  icon: Icon(Icons.delete, color: Colors.red),
                  onPressed: () {
                    // Delete the tag
                    provider.deleteTag(tag.id);
                  },
                ),
              );
            },
          );
        },
      ),
    );
  }
}
```

```

floatingActionButton: FloatingActionButton(
  onPressed: () {
    showDialog(
      context: context,
      builder: (context) => AddTagDialog(
        onAdd: (newTag) {
          Provider.of<ExpenseProvider>(context, listen: false).addTag(newTag);
          Navigator.pop(context); // Close the dialog after adding the new tag
        },
      ),
    );
  },
  tooltip: 'Add New Tag',
  child: Icon(Icons.add),
),
);
}
}

```

- `class TagManagementScreen extends StatelessWidget`: Defines a stateless widget named `TagManagementScreen`.
- Uses `Consumer<ExpenseProvider>` to listen to tag list updates.
- Implements a `ListView.builder` to render each tag as a list item.
- Defines a floating action button to show a dialog for adding a new tag.

The `tag management` screen should look like this:

←

Manage Tags

Breakfast

Lunch

Dinner

Treat

Cafe

Restaurant

Train

Vacation

Birthday

Diet

MovieNight

Tech

+

CarStuff

Step 8: Create the home screen

In this step, you will develop the home screen that lists all expenses. Each expense's information will be displayed, and you can navigate to the screen by tapping on an expense.

Create the HomeScreen class

- First, create the `HomeScreen` class.



dart



```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/expense_provider.dart';
import '../screens/add_expense_screen.dart';
import '../screens/category_management_screen.dart';
import '../screens/tag_management_screen.dart';
import 'package:intl/intl.dart';
import 'package:collection/collection.dart';
import '../models/expense.dart';
import '../models/expense_category.dart';

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen>
    with SingleTickerProviderStateMixin {
  late TabController _tabController;

  @override
  void initState() {
    super.initState();
    _tabController = TabController(length: 2, vsync: this);
  }

  @override
  void dispose() {
    _tabController.dispose();
    super.dispose();
  }
}
```

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("Expense Tracker"),
      backgroundColor: Colors.deepPurple[800],
      foregroundColor: Colors.white,
      bottom: TabBar(
        controller: _tabController,
        indicatorColor: Colors.white,
        labelColor: Colors.white,
        unselectedLabelColor: Colors.white70,
        tabs: [
          Tab(text: "By Date"),
          Tab(text: "By Category"),
        ],
      ),
    ),
    drawer: Drawer(
      child: ListView(
        padding: EdgeInsets.zero,
        children: <Widget>[
          DrawerHeader(
            decoration: BoxDecoration(color: Colors.deepPurple),
            child: Text('Menu',
              style: TextStyle(color: Colors.white, fontSize: 24)),
          ),
          ListTile(
            leading: Icon(Icons.category, color: Colors.deepPurple),
            title: Text('Manage Categories'),
            onTap: () {
              Navigator.pop(context); // This closes the drawer
              Navigator.pushNamed(context, '/manage_categories');
            },
          ),
          ListTile(
            leading: Icon(Icons.tag, color: Colors.deepPurple)

```

```

        onTap: () {
          Navigator.pop(context); // This closes the drawer
          Navigator.pushNamed(context, '/manage_tags');
        },
      ),
    ],
  ),
),
body: TabBarView(
  controller: _tabController,
  children: [
    buildExpensesByDate(context),
    buildExpensesByCategory(context),
  ],
),
floatingActionButton: FloatingActionButton(
  backgroundColor: Colors.deepPurple,
  onPressed: () => Navigator.push(context,
    MaterialPageRoute(builder: (context) => AddExpenseScreen())),
  tooltip: 'Add Expense',
  child: Icon(Icons.add),
),
);
}

```

```

Widget buildExpensesByDate(BuildContext context) {
  return Consumer<ExpenseProvider>(
    builder: (context, provider, child) {
      if (provider.expenses.isEmpty) {
        return Center(
          child: Text("Click the + button to record expenses.",
            style: TextStyle(color: Colors.grey[600], fontSize: 18)),
        );
      }
    },
  );
  return ListView.builder(
    itemCount: provider.expenses.length,

```

```

final expense = provider.expenses[index];
String formattedDate =
    DateFormat('MMM dd, yyyy').format(expense.date);
return Dismissible(
    key: Key(expense.id),
    direction: DismissDirection.endToStart,
    onDismissed: (direction) {
        provider.removeExpense(expense.id);
    },
    background: Container(
        color: Colors.red,
        padding: EdgeInsets.symmetric(horizontal: 20),
        alignment: Alignment.centerRight,
        child: Icon(Icons.delete, color: Colors.white),
    ),
    child: Card(
        color: Colors.purple[50],
        margin: EdgeInsets.symmetric(horizontal: 10, vertical: 12),
        child: ListTile(
            title: Text(expense.payee + " - \$" + expense.amount.toStringAsFixed(2)),
            subtitle: Text(
                "$formattedDate - Category: ${getCategoryNameById(context, expense.categoryId)}"),
            isThreeLine: true,
            trailing: Icon(Icons.edit, color: Colors.deepPurple),
            onTap: () => Navigator.push(
                context,
                MaterialPageRoute(
                    builder: (context) => AddExpenseScreen(initialExpense: expense),
                ),
            ),
        ),
    ),
);
}

```



```
}
```

```
Widget buildExpensesByCategory(BuildContext context) {  
  return Consumer<ExpenseProvider>(  
    builder: (context, provider, child) {  
      if (provider.expenses.isEmpty) {  
        return Center(  
          child: Text("Click the + button to record expenses.",  
            style: TextStyle(color: Colors.grey[600], fontSize: 18)),  
        );  
      }  
    },  
  );  
  
  // Grouping expenses by category  
  var grouped = groupBy(provider.expenses, (Expense e) => e.categoryId);  
  return ListView(  
    children: grouped.entries.map((entry) {  
      String categoryName = getCategoryNameById(  
        context, entry.key); // Ensure you implement this function  
      double total = entry.value.fold(  
        0.0, (double prev, Expense element) => prev + element.amount);  
      return Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: <Widget>[  
          Padding(  
            padding: EdgeInsets.all(8.0),  
            child: Text(  
              categoryName + " - Total: \$" + total.toStringAsFixed(2),  
              style: TextStyle(  
                fontSize: 18,  
                fontWeight: FontWeight.bold,  
                color: Colors.deepPurple,  
              ),  
            ),  
          ),  
          ListView.builder(  
            physics:
```

```

        shrinkWrap:
            true, // necessary to integrate a ListView within another ListView
        itemCount: entry.value.length,
        itemBuilder: (context, index) {
            Expense expense = entry.value[index];
            return ListTile(
                leading:
                    Icon(Icons.monetization_on, color: Colors.deepPurple),
                title: Text(expense.payee + " - \$" + expense.amount.toStringAsFixed(2)),
                subtitle: Text(DateFormat('MMM dd, yyyy')
                    .format(expense.date)),
            );
        },
    ),
],
);
}).toList(),
);
},
);
}

// home_screen.dart

String getCategoryNameById(BuildContext context, String categoryId) {
    var category = Provider.of<ExpenseProvider>(context, listen: false)
        .categories
        .firstWhere((cat) => cat.id == categoryId,
            orElse: () => ExpenseCategory(id: '', name: 'Unknown', isDefault: false));
    return category.name;
}
}

```

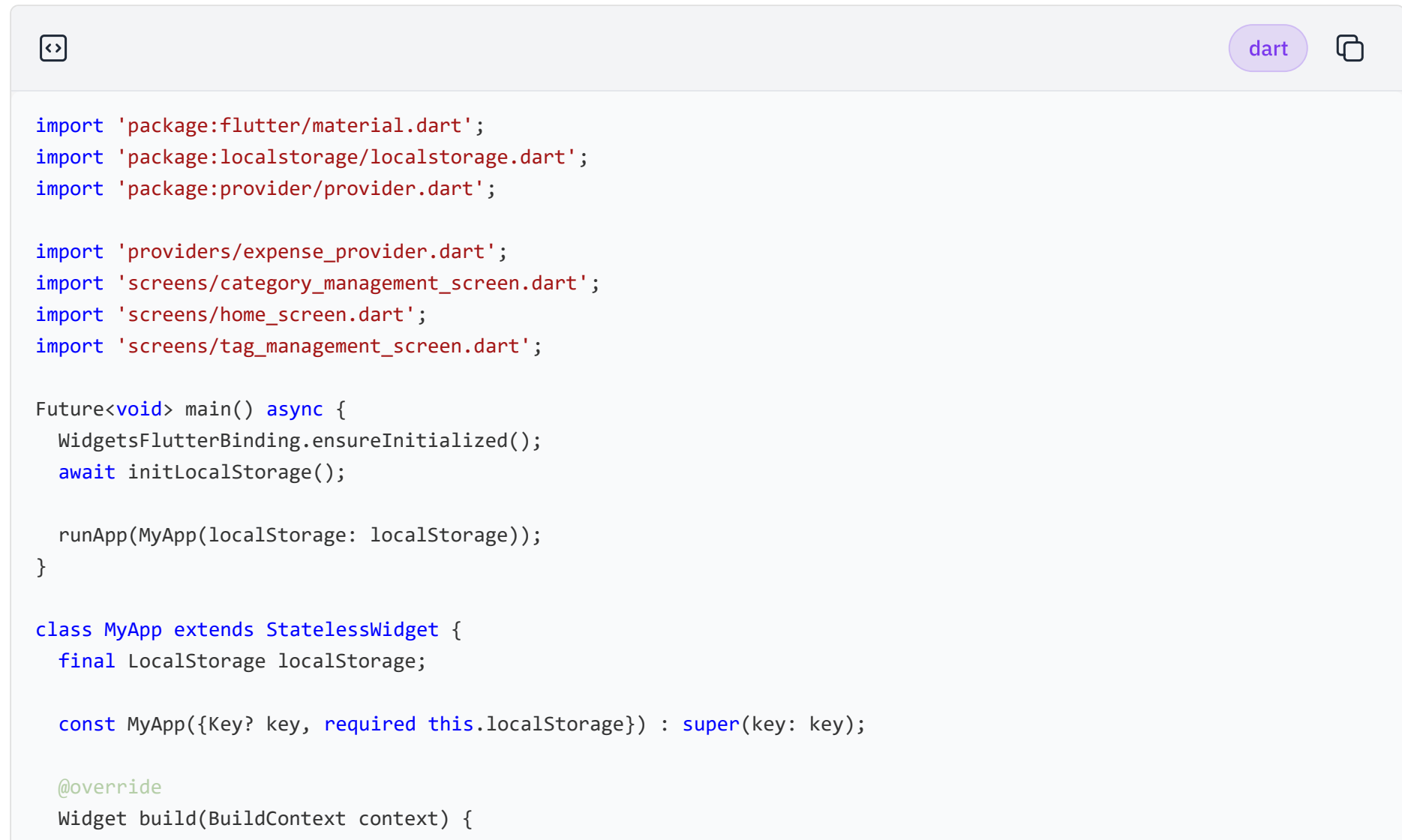
- `class HomeScreen extends StatelessWidget`: Defines a stateless widget named `HomeScreen`.

- Implements a `ListView.builder` to render each expense as a list item.
- Defines a floating action button to navigate to the `EditExpenseScreen` for adding a new expense.

The `home screen` has a few different states. When there is no expense, it should display the message "Add your first expense".

Step 9: Replace the main.dart file

Replace your main.dart file



The image shows a code editor window with a light gray header. On the left of the header is a code icon (two arrows pointing in opposite directions). On the right is a purple pill-shaped button with the text "dart" and a copy icon (two overlapping squares). The main area of the editor contains Dart code for a Flutter application. The code includes imports for Flutter Material, LocalStorage, Provider, Expense Provider, and two custom screens. It also shows the main function and the MyApp class, which extends StatelessWidget and uses LocalStorage.

```
import 'package:flutter/material.dart';
import 'package:localStorage/localstorage.dart';
import 'package:provider/provider.dart';

import 'providers/expense_provider.dart';
import 'screens/category_management_screen.dart';
import 'screens/home_screen.dart';
import 'screens/tag_management_screen.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initLocalStorage();

  runApp(MyApp(localStorage: localStorage));
}

class MyApp extends StatelessWidget {
  final LocalStorage localStorage;

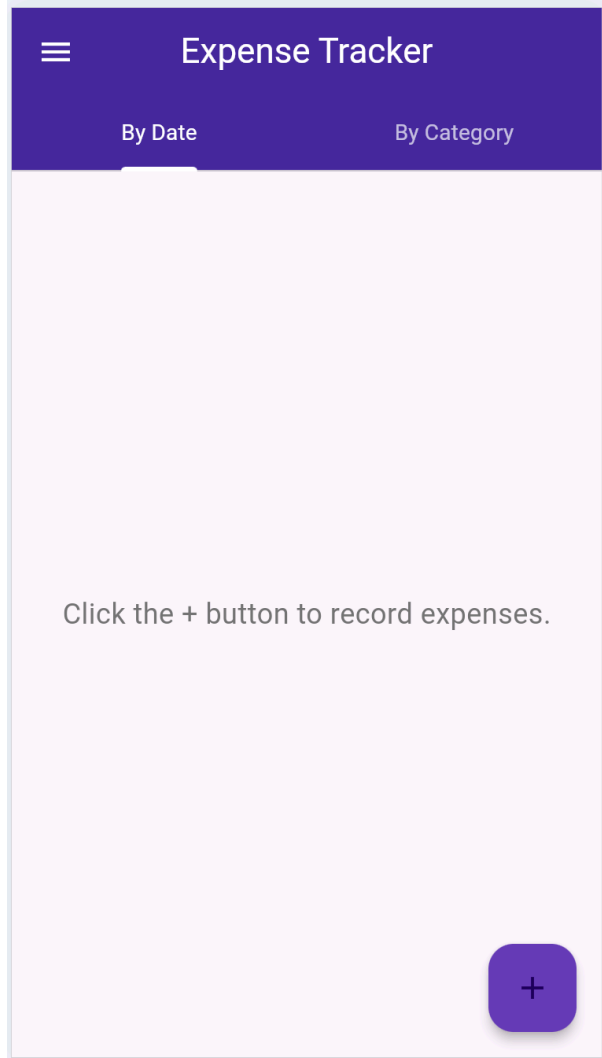
  const MyApp({Key? key, required this.localStorage}) : super(key: key);

  @override
  Widget build(BuildContext context) {
```

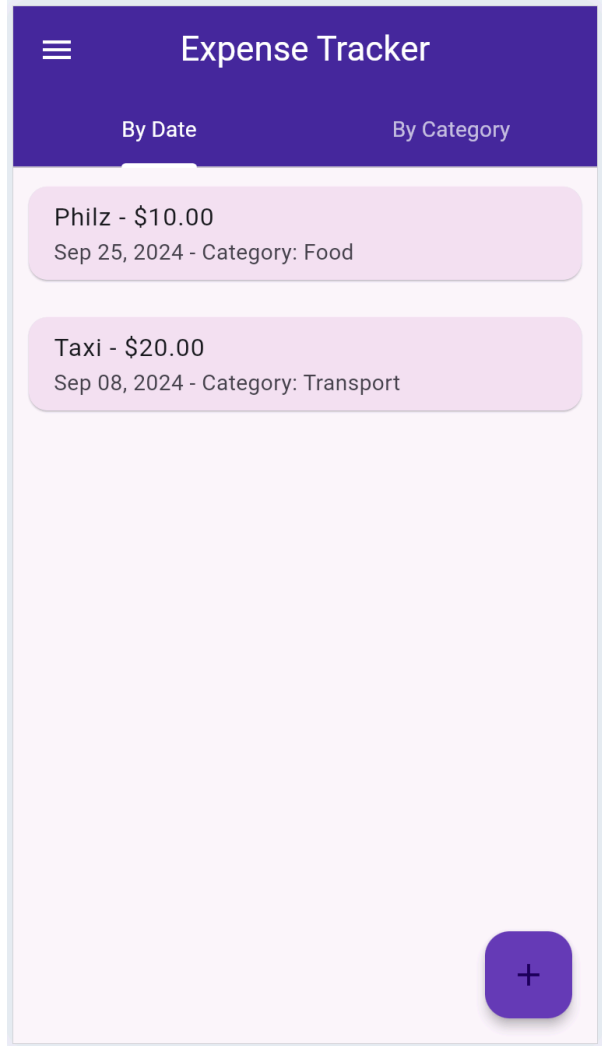
```

        ChangeNotifierProvider(create: (_) => ExpenseProvider(localStorage)),
    ],
    child: MaterialApp(
      title: 'Expense Tracker',
      debugShowCheckedModeBanner: false,
      initialRoute: '/',
      routes: {
        '/': (context) => HomeScreen(), // Main entry point, HomeScreen
        '/manage_categories': (context) =>
          CategoryManagementScreen(), // Route for managing categories
        '/manage_tags': (context) =>
          TagManagementScreen(), // Route for managing tags
      },
      // Removed 'home:' since 'initialRoute' is used to define the home route
    ),
  );
}
}

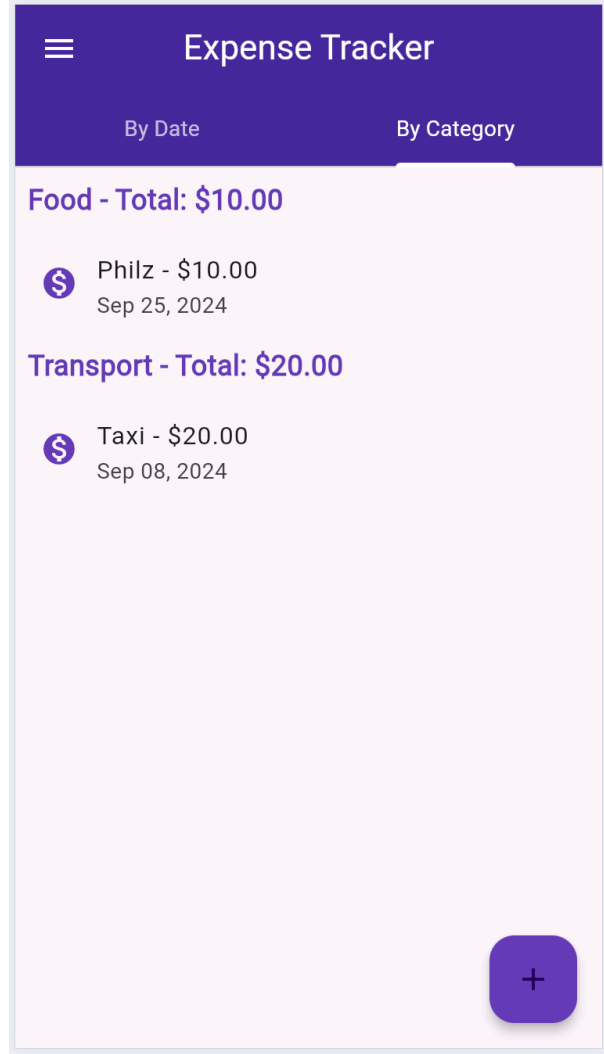
```



Once you have added one or more expenses, the first tab **By Date** should list out the expenses in the form of cards.



The **By Category** tab should group expenses by category.



Step 10: Run the lab

Flutter provides `hot reload` that makes it very easy to test the code as you are developing it. That functionality however does not work in the Cloud IDE. We have developed a `hot_reload.sh` script that essentially does the same thing. Please follow these steps to run the flutter application:

1. Change to the home directory

 bash 

```
cd /home/project
```

 Run

2. Set the `PROJECT_DIR` variable. This should be set to the `lib` directory of your flutter app folder.

 bash 

```
export PROJECT_DIR=/home/project/expense_manager/lib
```

 Run

3. Get the script and save to the `/home/project` folder.

 bash 

```
wget -O /home/project/hot_reload.sh https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/S3cw83zxb
```

 Run

4. Make the script executable by using the `chmod` command.

bash

```
chmod +x ./hot_reload.sh
```

Run

5. Run the command.

bash

```
./hot_reload.sh
```

Run

You should see output like this:

plaintext

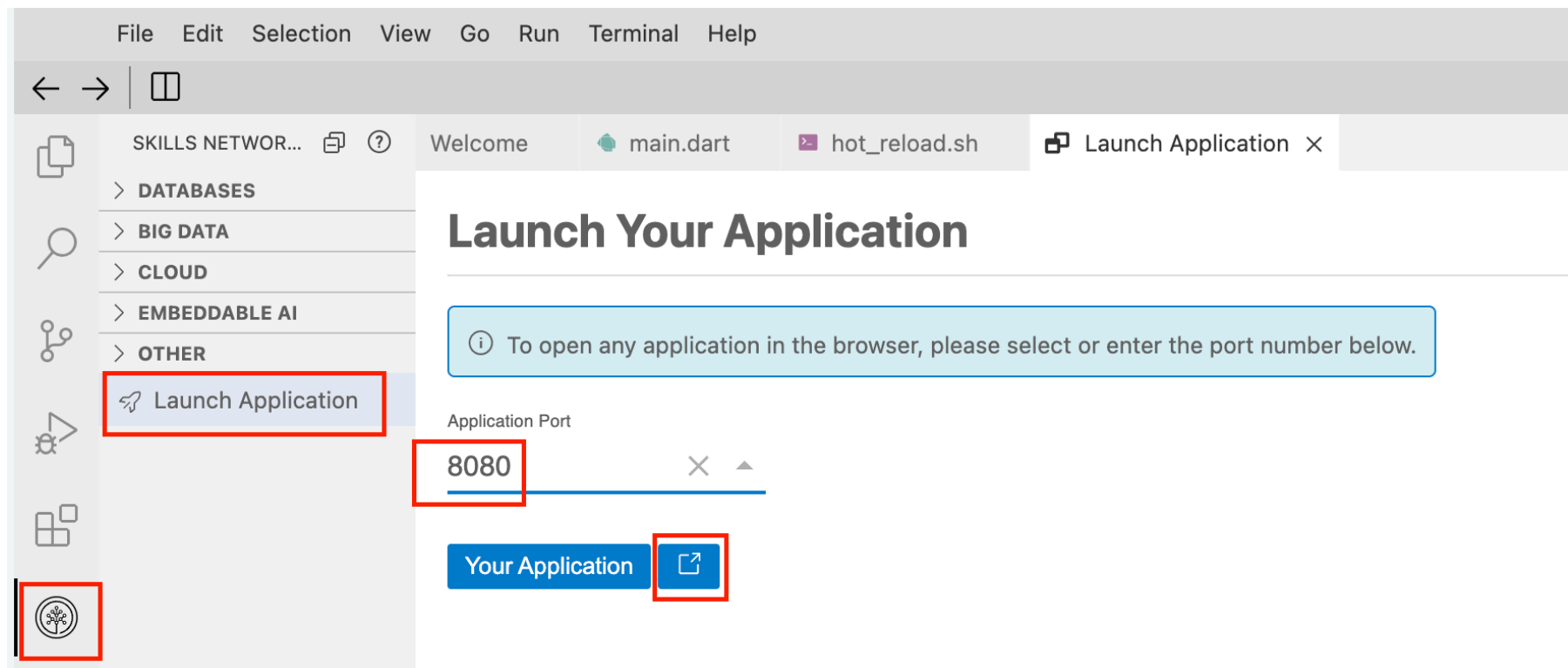
```
theia@theia-captainfedo1:/home/project$ ./hot_reload.sh

Using PROJECT_DIR: /home/project/expense_manager/lib
inotify-tools is already installed.
lsof is already installed.
Starting Flutter web server...
Port 8080 is free.

Compiling lib/main.dart for the Web... 984ms
✓ Built build/web
Flutter server started with PID 17675
Setting up watches. Beware: since -r was given, this may take a while!
Watches established.
```

6. Launch the application

- open the **Skills Network Toolbox** in the left bar of the Cloud IDE.
- click on **Launch Application** button
- enter **8080** as the port
- select the **launch in new window** to open the application in a new tab



7. Open developer tools to see the mobile view and look at local storage.

Complete solution

Here is the complete solution and set of files if you get stuck along the way.

1. lib/models/expense.dart

▶ [Click here for code](#)

2. lib/models/tag.dart

▶ [Click here for code](#)

3. lib/models/expense_category.dart

▶ [Click here for code](#)

4. lib/providers/expense_provider.dart

▶ [Click here for code](#)

5. lib/widgets/add_category_dialog.dart

▶ [Click here for code](#)

6. `lib/widgets/add_tag_dialog.dart`

▶ [Click here for code](#)

7. `lib/screens/add_expense_screen.dart`

▶ [Click here for code](#)

8. `lib/screens/category_management_screen.dart`

▶ [Click here for code](#)

9. `lib/screens/tag_management_screen.dart`

▶ [Click here for code](#)

10. `lib/screens/home_screen.dart`

▶ [Click here for code](#)

11. `lib/main.dart`

► [Click here for code](#)

12. pubspec.yaml

► [Click here for code](#)

Conclusion and next steps

This practice project focused on creating a basic expense management app. You can integrate local storage to ensure that the expenses' data persists even after the application restarts. This will involve modifying the `ExpenseProvider` to save and retrieve data from local storage.

Next steps

Congratulations on completing the expense management application with local storage! Here are some advanced features and improvements you can implement to enhance your application further:

- **Implement sorting and filtering**
 - Add functionality to sort expenses by date, amount, or category
 - Implement filters to view expenses by specific categories or date ranges
- **Add data visualization**
 - Implement charts to visualize expenses by category or monthly spending
 - Use a package like `charts_flutter` to create bar charts, pie charts, or line charts
- **Enhance UI/UX**
 - Improve the overall user interface and user experience
 - Add animations, transitions, and better error handling

- **Leverage APIs**

- Leverage a financial API to show the application in different currencies (USD, CAD, or INR)

Implementing these advanced features will create a more robust and user-friendly expense management application. Good luck, and happy coding!

Author(s)

Skills Network

© IBM Corporation. All rights reserved.

Practice Project: Create an Expense Tracking App

Estimated time: 60 minutes



Introduction

Welcome to this practice project, where you will develop an Expense Management application using Flutter. This project helps you apply the skills and knowledge acquired throughout the course.

The application you create will allow the user to manage their expenses with a friendly and intuitive user experience (UX). You will implement various essential features for an expense management app, such as adding, viewing, filtering, and deleting expenses.

Additionally, you will ensure that the data persists locally so that the user's expenses are not lost when the app is closed and reopened. We will use local storage for this feature.

User stories

The following user stories outline the key functionalities that you need to implement:

- "I want to view a list of all my expenses to see my spending history at a glance."
- "I want to add an expense with a payee, amount, notes, date, category, and tag to keep track of my spending."
- "I want to save my expenses locally so I do not lose them when I close and reopen the app."
- "I want to group my expenses by category to see how much I spend in different areas."
- "I want to delete an expense to remove any incorrect or unnecessary entries."
- "I want to manage categories and tags in the app settings."

Objectives

After completing this practice project, you will be able to:

- Create an intuitive UX for the user to list expenses
- Give the user a way to add and delete expenses
- Give the user a way to manage categories and tags
- Implement CRUD operations for expense data
- Integrate local storage to save data persistently

About Cloud IDE →